Kick Start 2019 - Round A

Analysis: Contention

The number of possible orderings of the requests is Factorial(**Q**), which would not be fast enough for either of the test sets. We can observe that for a chosen ordering of the requests, the number of seats that the system books in the last request does not depend on the ordering of the previous **Q** - 1 requests. So, we could start by finding the request to be processed last and move backwards towards the earlier requests. The answer is the minimum seats booked across the **Q** steps.

Another observation is that at every step, we can always choose this last request greedily from the remaining set: the one where we can book the maximum number of seats. An intuitive proof of why this works would be noticing that our final answer is non-increasing over the **Q** steps. Now, we can use exchange argument to prove this observation since choosing a request with lesser seats booked would not give us a better answer.

Test set 1 (Visible)

A naive implementation would be of the order O(**N** × **Q**), where we recalculate number of seats for remaining requests in O(**N**) using sweep line algorithm for each of the **Q** steps. We can speed this up with another observation: the number of unique ranges that the requests cover is at max 2 * **Q**, which would make our solution run in O($\mathbf{Q}^2$) for every test case.

Test set 2 (Hidden)

Let us try to speed up the slow process of recalculating number of seats we can book at every step For every seat, let us denote the *value* of a seat as the number of remaining requests trying to book this seat. Everytime the value of a seat drops to 1, we increase the number of seats we can book for the corresponding request containing this seat booking.

Since requests are represented by an interval, we can use an interval tree to support the operations of removing an interval after the initial construction of the tree. Each node of the interval tree stores the set of intervals that cover it, and also the minimum value of any seat in it's range. Whenever a remove operation makes the minimum value become one, we walk down the tree to find the seats that became one, and walk back up the tree to find out which interval is the only interval that now covers that seat. We now set that seat's value to infinity so that we don't process it again. This happens only once per seat, for a total amortised cost of O(**N**log**N**). In total this algorithm is O(**N**log(**Q**+**N**)), if you use a constant time set (like a hashset).

Kick Start 2019 - Round A

Analysis: Parcels

Test set 1 (Visible)

For Test Set 1, we are able to check all possible locations for the new delivery office in order to find the one that minimizes delivery time. We will do this in two stages. First, we will compute the delivery time for each square given the existing delivery offices. Second, we will try all possible locations for the new office. The precomputation in the first part will allow us to find the delivery time in the second part more efficiently.

For the first stage, we compute the delivery time of a square by iterating over the entire grid and finding the minimum manhattan distance to a square that has a delivery office. This has a time complexity of O(**RC**) per square for a total time complexity of $O((\mathbf{RC})^2)$.

For the second stage, we iterate over all possible locations for the new delivery office and for each location, search the entire grid for the square with the maximum new delivery time. The new delivery time is the minimum of the delivery time computed in the first part and the manhattan distance to the new delivery office. This also has a time complexity of O(**RC**) per delivery location for a total time complexity of $O((\mathbf{RC})^2)$, which is sufficient for Test Set 1.

Alternatively, we could skip the first stage if we use a faster way of computing the delivery time for each square such as breadth-first search. See the next section for more details.

Test set 2 (Hidden)

We can use a similar approach for Test Set 2, however we will need a more efficient way to compute the maximum delivery time for a new delivery location. We will do this by solving the following subproblem: given a value of **K**, can we add a new delivery office so that the maximum delivery time is at most **K**? The solution to this subproblem will be similar to Test Set 1, however having a target value of **K** allows us to identify exactly which squares need to be serviced by the new delivery office. We can use this difference to create a faster solution.

Note that if the answer to the original problem is **K**, then the answer to our subproblem will be 'No' for values in the range [1, **K**-1] and 'Yes' for values in the range [**K**, infinity]. For these kinds of subproblems, we can use binary search: if a given value works, then it is an upper bound for the answer; otherwise it's a strict lower bound for the answer. Hence, once we have a solution for the subproblem, we can use binary search to solve the original problem. This is a common technique to transform optimization problems into decision problems.

First, we efficiently compute the existing delivery time of every square by inverting the problem: instead of finding the shortest distance to a delivery office for each square, we find the shortest distance to each square from a delivery office. This can be done using a multiple-source, breadth-first search starting at all of the delivery offices. A multiple-source BFS is the same as a regular BFS except that you use multiple starting locations instead of one. This search visits each square at most once, which gives us a time complexity of O(**RC**).

Second, we identify all of the squares which have a delivery time greater than **K** and then determine if there exists a location that is within a distance of **K** to each of these squares. In order to do this efficiently, we note that the manhattan distance has an equivalent formula:

$$\text{dist}((x_1, y_1), (x_2, y_2)) = \max(\text{abs}(x_1 + y_1 - (x_2 + y_2)), \text{abs}(x_1 - y_1 - (x_2 - y_2)))$$

This formula is based on the fact that for any point, the set of points within a manhattan distance of **K** form a square rotated by 45 degrees. The benefit of this formula is that if we fix (x_2, y_2), the distance will be maximized when $x_1 + y_1$ and $x_1 - y_1$ are either maximized or minimized.

Hence, we can compute the maximum and minimum values of both $x_1 + y_1$ and $x_1 - y_1$ for all squares with a delivery time greater than **K**. Then, we can try all possible locations for the new delivery office and check if the maximum distance from the location to a square with a current delivery time greater than **K** is at most **K** in constant time. Hence, we can check if the answer is at most **K** with a time complexity of O(**RC**).

With the binary search, the time complexity becomes O(**RC**log(**R**+**C**)), which is sufficient for the test set. There is a way to improve this to O(**RC**) time by computing the min/max values mentioned above for all possible **K** in a single pass over the grid and then using casework to determine if a viable new delivery office location exists for each **K**, but this optimization is unnecessary.

Kick Start 2019 - Round A

Analysis: Training

To make a fair team, we have to train the members of the team to the same skill level as the most skillful member of the team.
For any **P** students we pick, the time required to make a fair team is = $\Sigma(\max(\mathbf{S}_i, \mathbf{S}_{i+1} \ldots \mathbf{S}_P) - \mathbf{S}_i)$, for all students i = 1 to **P** in the team. Our goal is to minimize this value.
One possible approach could be to go through all possible subsets of **P** students, from the given **N** students. But there exists ${}^{\mathbf{N}}C_{\mathbf{P}}$ such subsets(here symbol C denotes Combination). Number of such subsets will be in the order of Factorial(**N**) and so enumerating through all of them will not fit within the time limit.

Test set 1 (Visible)

We can start with the observation that once we fix the student with highest skill level x, to minimize our goal we should always choose students with skills as high as possible, but less than or equal to x. Hence we can sort students on the basis of skill level in decreasing order, and iterate over each student assuming they would have the highest skill level in the team. Say, this student is at position i in the sorted sequence; the team would be formed of students on positions i, i + 1, ..., i + **P** - 1 (i.e. a contiguous subarray of size **P**).
For each subarray of size **P** in the sorted array, we can calculate the training time required to make a fair team using the aforementioned formula. There are **N** - **P** + 1 subarrays of size **P**, and the complexity of calculating training time of subarray size **P** is O(**P**). So the overall complexity of this approach is O(**N** × **P**), which will be sufficient for test set 1.

Test set 2 (Hidden)

We need to go through all of the subarrays once, but can we calculate the training time of a subarray faster?
Let us assume the array is sorted in decreasing order. The training time formula for a subarray starting at position i is
= $\Sigma(\mathbf{S}[i] - \mathbf{S}[j])$ where j = i to i + **P** -1
= **P** × **S**[i] - $\Sigma(\mathbf{S}[j])$ where j = i to i + **P** - 1
As we always need the sum of a contiguous subarray, we can pre compute the prefix sum of the whole array in advance, and get the sum of any subarray in O(1) time, which makes the calculation of training time O(1).
So, the overall complexity of this approach is O(**N**).